

Zalord — Hệ thống Chat Microservices

Kiến trúc & Triển khai

Nhóm 17

Trường Đại học Công nghệ Thông tin — ĐHQG TP.HCM
GVHD: TS. Nguyễn Duy Khánh

Tháng 6, 2026

Nội dung trình bày

- 1 Tổng quan hệ thống
- 2 Domain-Driven Design
- 3 Nguyên lý thiết kế Microservices
- 4 Giao tiếp giữa các dịch vụ
- 5 Thiết kế & Quản lý cơ sở dữ liệu
- 6 Triển khai Docker & Kubernetes
- 7 API Gateway
- 8 Authentication & Authorization
- 9 Logging, Monitoring, Tracing
- 10 CI/CD & Triển khai
- 11 Resiliency Scenarios
- 12 Kết luận

Giới thiệu Zalord

Zalord là gì?

- Ứng dụng chat real-time kiến trúc **Microservices**
- **7 services** độc lập, polyglot Java / Go
- Hỗ trợ: nhắn tin 1-1, nhóm, file đính kèm, real-time presence
- Frontend: React 19 + TypeScript + Vite + WebSocket

Tech stack tổng quan

- **Gateway:** Kong 3.7 (DB-less declarative)
- **Messaging:** RabbitMQ / Kafka (switchable)
- **Storage:** PostgreSQL ×6, Redis, MinIO (S3)
- **Observability:** Prometheus + Grafana
- **Deploy:** Docker Compose (local) / k3s VPS (dev) / GKE (prod)
- **CI/CD:** GitHub Actions → Docker Hub → GKE / k3s

Danh sách Microservices

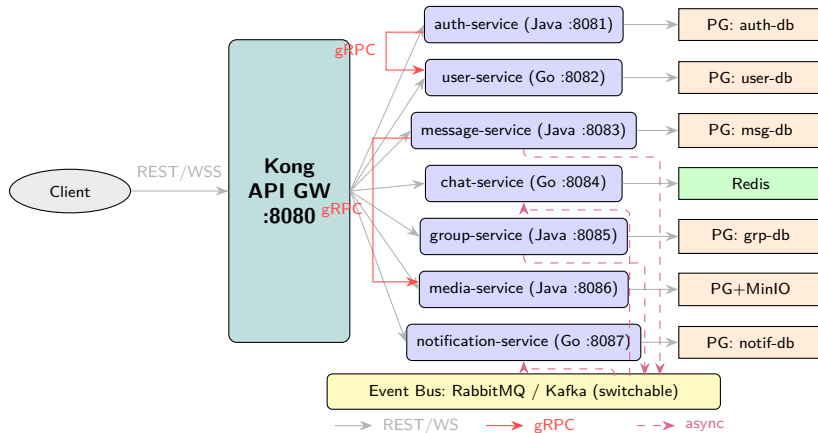
Service	Lang	Port
auth-service	Java/Spring	8081
user-service	Go	8082
message-service	Java/Spring	8083
chat-service	Go	8084
group-service	Java/Spring	8085
media-service	Java/Spring	8086
notification-service	Go	8087

gRPC Internal (không qua Kong)

auth → user (CreateProfile :9082)

message → media (ValidateAttachments :9086)

Kiến trúc tổng quan



Tại sao không dùng Modulithic? (Req 1)

Lý do chọn Microservices

- **Polyglot language:** chat-service dùng Go (goroutines) cho WebSocket fan-out hiệu năng cao; business logic dùng Java Spring Boot
- **Independent scaling:** chat-service cần scale riêng khi traffic tăng, không kéo theo auth-service
- **Independent deploy:** cập nhật media-service không ảnh hưởng chat real-time
- **Fault isolation:** media-service lỗi không kéo sập toàn bộ hệ thống

Nếu là Modulithic...

- Không thể mix Go + Java trong 1 process
- Scale toàn bộ khi chỉ cần scale 1 module
- Deploy toàn bộ khi chỉ thay đổi 1 module
- 1 memory leak / goroutine leak làm sập tất cả

Tổ chức Code

Tại sao chọn Layered thay vì Clean Architecture hay Vertical Slice?

Tiêu chí	Layered (Đang dùng)	Clean / Hexagonal	Vertical Slice (VSA)
Số file / tính năng	3 file (Controller, Service, Repo)	5-7 file (Port, Adapter, Use-case, Entity, DTO)	3-4 file (Request, Response, Handler inline)
Share Dependencies (VD: Config, Repo)	Dễ dàng tiêm 1 lần vào chung class Service	Dễ dàng tiêm vào chung Use case class	Khó — mỗi slice bị cô lập, phải tự tiêm lại từ đầu
Độ tương thích với Framework	Tuyệt đối (Hưởng lợi từ Spring annotations)	Xung đột (Domain không được chứa thư viện Spring/JPA)	Kém (Spring không hỗ trợ MediatR built-in như .NET)
Độ phức tạp & Duplication	Thấp, code base đơn giản, tập trung	Boilerplate quá cao so với quy mô dự án 3 tháng	Trùng lặp code nhiều (VD: Outbox pattern phải copy)

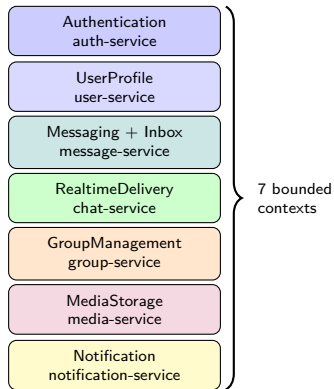
Kết luận cho bài toán thực tế của Zalord

Team 3 người, thời hạn 3 tháng, service nhỏ (~8-10 endpoints) và không có nhu cầu đổi Database.

→ **Layered Architecture** mang lại tốc độ code nhanh nhất, tận dụng tối đa tiện ích của Spring Boot, không bị "over-engineering".

Phân chia theo DDD — Bounded Contexts (Req 2)

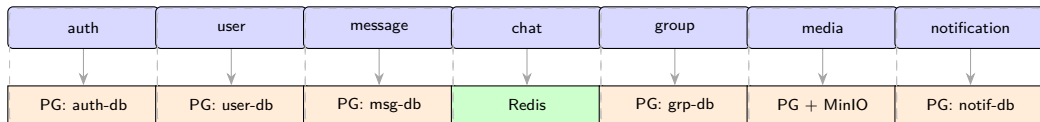
Service	Bounded Context	Aggregate Root
auth-service	Authentication	Account, Session
user-service	UserProfile	UserProfile
message-service	Messaging + Inbox	Message, ConversationView
chat-service	RealtimeDelivery	Connection
group-service	GroupManagement	Group
media-service	MediaStorage	MediaItem
notification-service	Notification	Notification



DB per Bounded Context (Req 2)

Nguyên tắc phân chia

- Mỗi service có **1 bounded context** độc lập
- Không share entity/table giữa các service
- Giao tiếp qua **API / Event** (không gọi thẳng DB)
- conversation.id được tái sử dụng làm group.id (shared key, không shared schema)



Mỗi service chỉ truy cập database của chính mình — không share schema

Redis (shared): auth sessions, chat presence/pub-sub,
message membership cache

MinIO (shared): avatars,
attachments

Single Responsibility Principle (Req 3)

SRP — Mỗi service 1 trách nhiệm rõ ràng

- **chat-service**: CHỈ quản lý WebSocket connections + fan-out messages → không write business DB, không có business logic
- **media-service**: CHỈ quản lý upload lifecycle (generate presigned URL → MinIO → finalize metadata)
- **notification-service**: CHỈ consume events + persist + serve notification list
- **auth-service**: CHỈ authentication (issue/validate token, session management)

High Cohesion & Low Coupling (Req 3)

High Cohesion — Low Coupling

- **High Cohesion:** dữ liệu và logic cùng domain nằm trong 1 service, không scatter
- **Low Coupling:** chỉ **2 synchronous dependencies** (gRPC):
 - auth → user (registration)
 - message → media (validate attachments)
- Mọi giao tiếp còn lại: **async event bus**
- Downstream services không parse JWT — trust header từ Kong

Execution Pipeline

Request → Kong (validate) → Service → DB / Event Bus

Không có cross-service synchronous chain dài hơn 2 hops

Sync vs Async — Khi nào dùng gì? (Req 4)

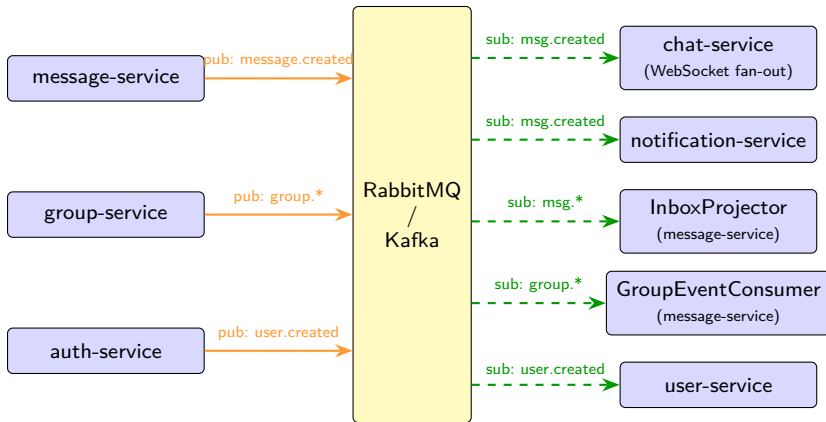
Synchronous — gRPC (2 trường hợp)

- **auth** → **user-service** (CreateProfile):
 - Cần response ngay để rollback nếu profile creation fail
 - Deadline 5 giây, circuit breaker Resilience4j
- **message** → **media-service** (ValidateAttachments):
 - Validate media IDs trước khi persist message
 - Deadline 3 giây (hot path user-facing)
- Dùng gRPC vì: type-safe contract (Protobuf), binary encoding nhanh hơn REST, không qua Kong (internal network)

Asynchronous — Event Bus (Kafka / RabbitMQ)

- **message.created** → chat-service (WebSocket fan-out), notification-service, InboxProjector
- **group.*** (created/member-added/removed/updated) → message-service (project Conversation)
- **user.created** → user-service
- Dùng async vì: fan-out 1-to-many, producers không cần chờ consumers, failure isolation
- **Switchable**: EVENT_BUS=rabbitmq hoặc kafka — cùng interface EventBus (Factory pattern)

Event-Driven Architecture — Message Flow (Req 4)



Benchmark: Kafka vs RabbitMQ

Kết quả Benchmark

- **Kafka:** Throughput cao hơn (hàng trăm ngàn msg/s), phù hợp cho log aggregation và event sourcing.
- **RabbitMQ:** Độ trễ (latency) thấp hơn ở mức tải trung bình, hỗ trợ routing linh hoạt (topic exchange, dead-letter queue).

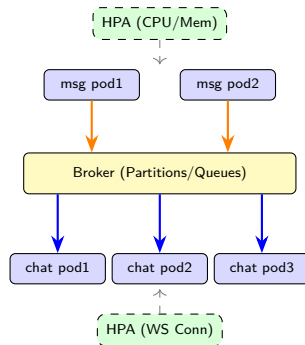
[PLACEHOLDER — biểu đồ benchmark]

Biểu đồ so sánh Throughput & Latency giữa Kafka và RabbitMQ

Scaling cho luồng gửi Message

Chiến lược Scaling

- **Producer (message-service):** Scale theo CPU/Memory (HPA) vì việc lưu DB và publish event đòi hỏi xử lý nhanh.
- **Consumer (chat-service):** Scale theo số lượng kết nối WebSocket đang mở, giới hạn số lượng connection per pod.
- **Broker:** Tăng số lượng Partition (Kafka) hoặc chia nhỏ Queue (RabbitMQ) để scale việc tiêu thụ event.



Thiết kế Endpoint & Swagger (Req 5)

REST API — Naming Convention

- Base path: `/api/v1/{resource}` (plural noun)
- POST `/api/v1/auth/register`
- GET `/api/v1/users`
- GET `/api/v1/conversations`
- POST `/api/v1/messages`
- POST `/api/v1/groups/{id}/members`
- DELETE `/api/v1/groups/{id}/members/{uid}`
- POST `/api/v1/media/upload-url`
- WebSocket: `/ws/chat?token={jwt}`

Error Response

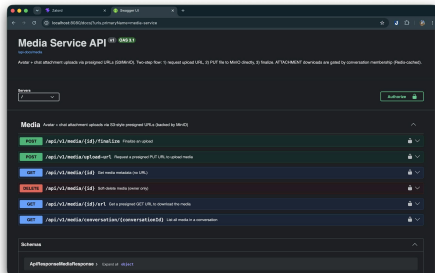
Chuẩn **RFC 7807**
(Platform-agnostic):

```
{"type":...,  
  "status":400,  
  "detail":"...",  
  "title":"..."}
```


Swagger / OpenAPI Aggregation (Req 5)

Swagger / OpenAPI Aggregation

- Java services: **springdoc-openapi** → /v3/api-docs
- Go services: **swaggo** → /v3/api-docs
- Kong route /api-docs/{svc} → strip_path: true → forward đến service /v3/api-docs
- **Swagger UI** container tại /docs aggregate tất cả



DB per Service & Consistency (Req 6)

DB per Service

- 6 PostgreSQL instances độc lập + Redis + MinIO
- Không service nào truy cập trực tiếp DB của service khác
- **Strong consistency**: trong 1 service (ACID transactions)
- **Eventual consistency**: cross-service qua event bus
 - group member change → message-service projection có độ trễ nhỏ
 - message.created → inbox view update không đồng thời

Consistency cần thiết ở đâu?

- **Strong**: auth (login/register), message persist (không mất tin nhắn)
- **Eventual**: inbox unread count, notification, presence status

CAP Theorem & Saga Pattern (Req 6)

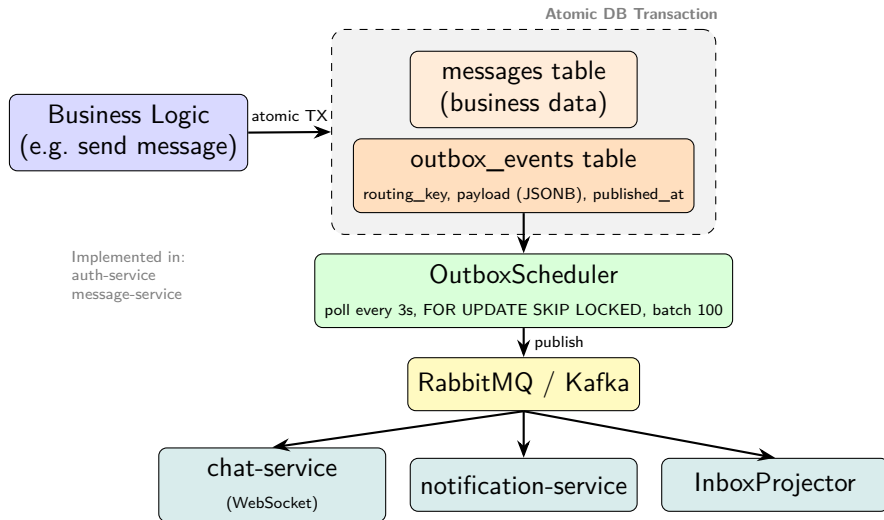
CAP Theorem

- PostgreSQL: **CP** (Consistency + Partition Tolerance)
- Chấp nhận availability giảm khi partition
- Redis: **AP** — eventual consistency cho presence data

Vì sao không dùng Saga Pattern?

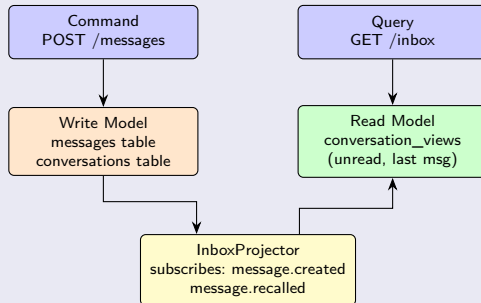
Message persist là local transaction (ghi vào 1 DB của message-service). Distributed transaction duy nhất là Registration (auth → user) — chỉ gồm 2 bước nên dùng gRPC + Circuit Breaker (Resilience4j) là đủ an toàn, không cần độ phức tạp của Saga.

Transactional Outbox Pattern (Req 7)



CQRS trong message-service (Req 7)

CQRS trong message-service

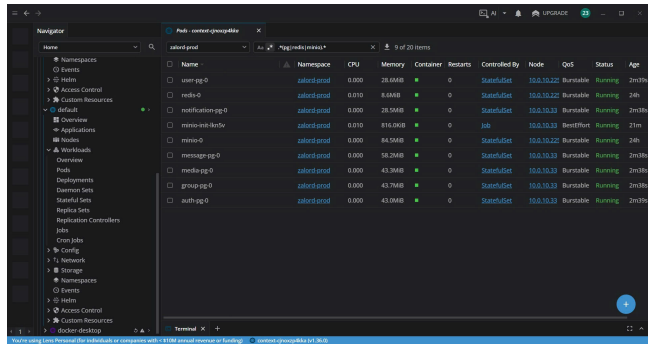


projection có độ trễ nhỏ (eventual consistency)

Polyglot Persistence (Req 7)

Polyglot Persistence

Store	Dùng cho
PostgreSQL ×6	Business data (ACID)
Redis	Sessions, presence, pub/sub
MinIO (S3)	Avatars, attachments



The screenshot shows the Zalard dashboard interface. On the left is a sidebar with a 'Navigator' menu containing categories like Namespaces, Events, Helm, Access Control, Custom Resources, default (selected), Overview, Applications, Nodes, Workloads, Deployments, Daemon Sets, Stateful Sets, Replica Sets, Replication Controllers, Jobs, Cron Jobs, Config, Network, Storage, Namespaces, Events, Helm, Access Control, and Custom Resources. The main panel displays a table of pods in the 'zalord-prod' namespace. The table has columns for Name, Namespace, CPU, Memory, Container, Restarts, Controlled By, Node, QoS, Status, and Age. There are 9 items listed, all with a status of 'Running'.

Name	Namespace	CPU	Memory	Container	Restarts	Controlled By	Node	QoS	Status	Age
user-pg-0	zalord-prod	0.00%	28.6MB		0	StatefulSet	10.0.10.22	Burstable	Running	2m39s
redis-0	zalord-prod	0.01%	8.6MB		0	StatefulSet	10.0.10.22	Burstable	Running	24h
notification-pg-0	zalord-prod	0.00%	28.5MB		0	StatefulSet	10.0.10.33	Burstable	Running	2m38s
minio-init-4n5v	zalord-prod	0.01%	816.0KB		0	Job	10.0.10.33	BestEffort	Running	21m
minio-0	zalord-prod	0.00%	84.5MB		0	StatefulSet	10.0.10.22	Burstable	Running	24h
message-pg-0	zalord-prod	0.00%	58.2MB		0	StatefulSet	10.0.10.33	Burstable	Running	2m38s
media-pg-0	zalord-prod	0.00%	43.3MB		0	StatefulSet	10.0.10.33	Burstable	Running	2m38s
group-pg-0	zalord-prod	0.00%	43.7MB		0	StatefulSet	10.0.10.33	Burstable	Running	2m38s
auth-pg-0	zalord-prod	0.00%	43.0MB		0	StatefulSet	10.0.10.33	Burstable	Running	2m39s

Dockerfile & Docker Compose (Req 8)

Multi-Stage Dockerfile — Java (auth-service)

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-jammy AS build
WORKDIR /workspace
COPY mvnw . && COPY .mvn .mvn
COPY pom.xml .
RUN chmod +x mvnw && ./mvnw -B dependency:go-offline
# Layer cache: deps downloaded before source copy
COPY src src
RUN ./mvnw -B -DskipTests clean package \
    && java -Djarmode=layertools -jar target/*.jar \
    extract --destination target/extracted

# Stage 2: Runtime (JRE only, alpine = small)
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring
COPY --from=build /workspace/target/extracted/ ./
EXPOSE 8081
```

Multi-Stage Dockerfile — Go (chat-service)

```
# Stage 1: Build
FROM golang:1.26.1-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download          # deps cached separately
COPY . .
RUN CGO_ENABLED=0 go build -o server ./cmd/server

# Stage 2: Runtime (scratch-like, ~8MB)
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/server .
EXPOSE 8084
CMD ["/server"]
```

Docker Compose (Req 8)

Docker Compose

- 20+ containers, 1 network zalord-net
- Named volumes per DB (pgdata_auth, ..., miniodata, redisdata)
- Env vars từ .env file, không hardcode

Kustomize Overlays

- Path: `deploy/k8s/overlays/dev/`
- Namespace: `zalord-dev`
- **Workloads:** 7 Deployments (services)
- **StatefulSets:** Kafka, RabbitMQ, Redis, PostgreSQL×6, MinIO (kèm Persistent Volumes 1–5Gi)
- **Resources:** req/limit (e.g. auth: 100m/384Mi req)
- **Probes:** TCP liveness, HTTP `/health`
- **Secrets:** SOPS + Age encryption

[PLACEHOLDER — chụp hình]

```
kubectl get pods -n zalord-dev
```

*Chụp màn hình tất cả pods
ở trạng thái Running*

[PLACEHOLDER — chụp hình]

Lens / K9s IDE hoặc

```
kubectl get all -n zalord-dev
```


hiển thị cluster overview

Quản lý Cluster với K8s Lens

K8s Lens / OpenLens

- Sử dụng **K8s Lens** làm công cụ quản lý cluster trực quan (GUI).
- Dễ dàng theo dõi trạng thái Pods, Nodes, Deployments, và StatefulSets.
- Hỗ trợ xem logs, exec vào container, và chỉnh sửa YAML trực tiếp không cần thao tác dòng lệnh.

[PLACEHOLDER — chụp hình]

*Chụp màn hình K8s Lens
đang quản lý cluster zalord-dev*

Service Discovery (Req 9)

Service Discovery — K8s DNS

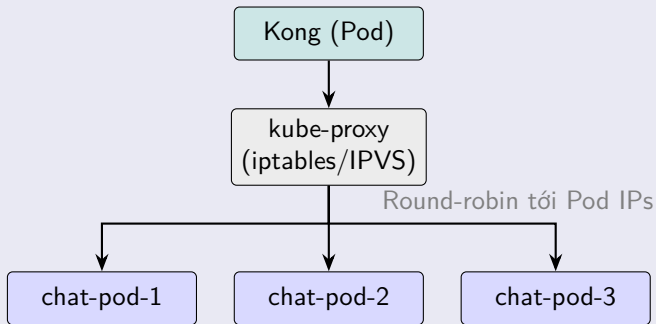
- K8s tự tạo DNS entry cho mỗi ClusterIP Service
- `auth-service.zalord-dev.svc.cluster.local`
- Các service gọi nhau qua tên service nội bộ (e.g. `http://user-service:8082`)
- **KHÔNG cần Eureka hay Consul** — K8s DNS thay thế hoàn toàn

Cấu hình gRPC target

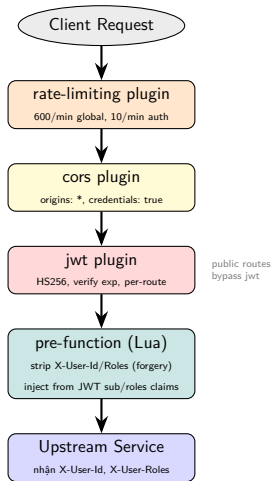
- `ZALORD_USER_SERVICE_GRPC_TARGET=user-service:9082`
- `ZALORD_MEDIA_SERVICE_GRPC_TARGET=media-service:9086`
- Quá trình: DNS resolve → ClusterIP → Pod IP

Load Balancing (Req 9)

Load Balancing — Server-side



Kong API Gateway — Filter Chain (Req 10)



Các tính năng Kong đã cấu hình

- **Rate Limiting**: 600 req/min per IP (global), 10 req/min (auth routes /login, /register)
- **CORS**: global, tất cả origins (dev), credentials enabled
- **JWT Filter**: validate HS256, kiểm tra exp claim
- **Identity Injection**: Lua pre-function inject X-User-Id (từ sub) và X-User-Roles (từ roles array)
- **Prometheus**: metrics tại admin :8001/metrics

Kong — DB-less Mode & Route Table (Req 10)

Declarative Config (DB-less)

- Config tại `infra/kong/kong.yml` (version control). **Cùng 1 file** cho Dev & Prod (tránh config drift).
- `__JWT_SECRET__` được thay bằng env var khi start — bảo mật secret.
- **KHÔNG dùng K8s Ingress Controller** (`networking.k8s.io/v1`). Đây là **lựa chọn thiết kế** standalone (xem `EXPLANATION.md`).

Route Table

Route	Upstream	JWT
<code>/api/v1/auth/*</code>	<code>auth:8081</code>	×
<code>/api/v1/users</code>	<code>user:8082</code>	✓
<code>/api/v1/messages</code>	<code>message:8083</code>	✓
<code>/ws/chat</code>	<code>chat:8084</code>	✓
<code>/api/v1/groups</code>	<code>group-service:8085</code>	✓
<code>/api/v1/media</code>	<code>media-service:8086</code>	✓
<code>/api/v1/notifications</code>	<code>notification-svc:8087</code>	✓
<code>/docs</code>	<code>swagger-ui:8080</code>	×

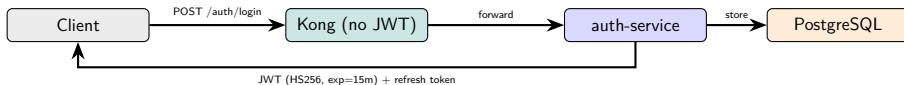
[PLACEHOLDER — chụp hình]

Kong Admin API:
`curl localhost:8001/routes`

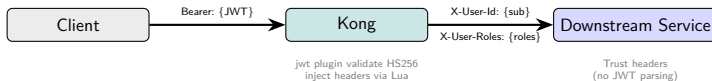
hoặc Kong Manager UI
hiển thị danh sách routes/plugins

JWT Authentication Flow (Req 11)

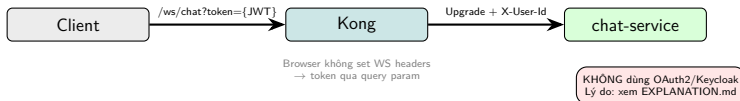
(1) Login



(2) API Call



(3) WebSocket



ConfigMap — Non-sensitive config (Req 12)

ConfigMap — Non-sensitive config

- **Shared:** shared-config.yaml
 - EVENT_BUS=rabbitmq
 - RABBITMQ_HOST=rabbitmq
 - KAFKA_BOOTSTRAP=kafka:9092
 - REDIS_HOST=redis
 - JWT_EXPIRY_MINUTES=15
- **Per-service:** {svc}-config.yaml — database host/port/name, gRPC targets, service-specific config
- Mount as env vars vào Deployment

Tại sao cần ConfigMap?

- Thay đổi config không cần build lại Docker image
- Khác nhau giữa environments (dev/staging/prod) qua Kustomize overlays

Secret — Sensitive data (Req 12)

Secret — Sensitive data (SOPS encrypted)

- **Shared:** DB passwords, JWT_SECRET, MQ credentials
- **Per-service:** database URLs với credentials
- Files: *.secret.enc.yaml — encrypted với **SOPS + Age key**
- Age key lưu trong **GitHub Actions Secret**
- CI/CD decrypt tại deploy time: `sops -d file.enc.yaml`
- Secret encrypted at rest trong Git — an toàn commit public repo

Kong JWT Secret

kong.yml chứa `__JWT_SECRET__` placeholder. Container entrypoint dùng `sed` substitute từ K8s Secret env var. Secret không bao giờ nằm trong file config.

Prometheus & Grafana — Observability (Req 13)

Prometheus Scrape Config

Target	Endpoint
Java Services	/actuator/prometheus
Go Services	/metrics
Kong	kong:8001/metrics
RabbitMQ	rabbitmq:15692/metrics
Kafka	kafka-exporter:9308

[PLACEHOLDER — chụp Grafana]

Dashboard: zalord-overview.json
http://localhost:3000

Chụp panels:

- HTTP request rate/latency
- Active WS connections
- Message throughput

Metrics được track

- HTTP latency, request count, status codes
- Active WebSocket connections (chat)
- Message throughput, queue lag
- JVM heap, GC, Pod resource

[PLACEHOLDER — chụp Prom]

Prometheus targets page
localhost:9090/targets
all targets: State=UP

Logging Tập Trung với Loki

Grafana Loki

- Thay vì dùng stack ELK nặng nề, hệ thống dùng **Loki** + **Promtail** nhẹ hơn nhiều, tối ưu cho K8s.
- Promtail tự động thu thập stdout của tất cả pods và đẩy lên Loki.
- Truy vấn log tập trung thông qua **Grafana** (LogQL). Không cần login vào từng pod bằng `kubectl logs`.

[PLACEHOLDER — chụp hình Grafana/Loki]

Chụp màn hình Grafana Explore đang query log của một service sử dụng Loki LogQL

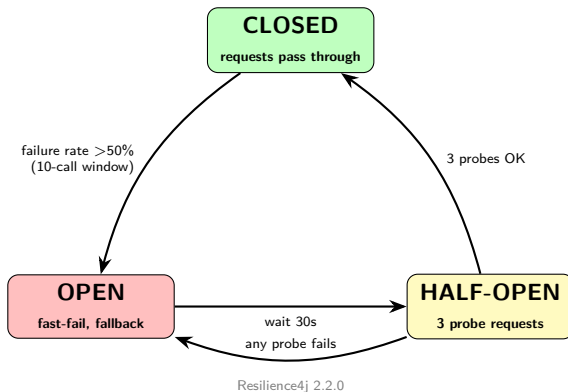
OpenTelemetry & Jaeger

- Sử dụng **OpenTelemetry (OTel)** để inject trace-id qua các HTTP header và gRPC metadata.
- **Luồng:** API Gateway → Microservice A → gRPC → Microservice B → RabbitMQ/Kafka → Consumer C.
- Trace-id được xuyên suốt qua cả giao tiếp đồng bộ (REST/gRPC) và bất đồng bộ (Event Bus).
- Dữ liệu được đẩy về **Jaeger** UI để trực quan hóa toàn bộ chuỗi request.

[PLACEHOLDER — sơ đồ & chụp hình]

*Sơ đồ luồng OTel tracing
và chụp màn hình Jaeger UI
hiển thị một span hoàn chỉnh*

Circuit Breaker Diagram (Req 14)



Timeout, Circuit Breaker & Recovery (Req 14)

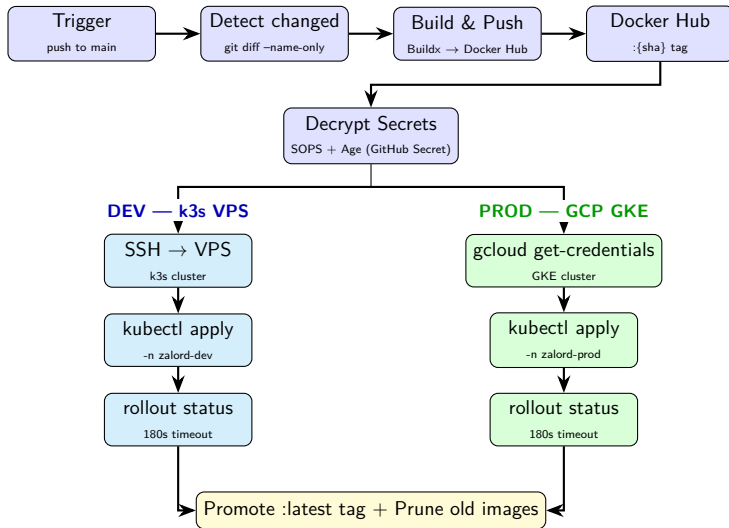
Implementation

- **auth-service** → user-service gRPC:
 - `@CircuitBreaker(name="userGrpc")`
 - gRPC deadline: **5 giây**
 - Fallback: `createProfileFallback()`
- **message-service** → media-service gRPC:
 - `@CircuitBreaker(name="mediaGrpc")`
 - gRPC deadline: **3 giây** (hot path)
 - Fallback: `validateFallback()`

Go services — Error Handling

- **PermanentError**: drop message (ack), log error
- **Transient error**: nack + requeue → RabbitMQ retry
- Frontend WebSocket: exponential backoff reconnect (cap 15s)

GitHub Actions Pipeline (Req 15)



GitHub Actions Pipeline (Req 15) — Screenshot

[PLACEHOLDER — chụp hình]

GitHub Actions → tab Actions → chọn workflow run thành công → chụp màn hình các steps

Versioning & Rollback Strategy (Req 15) — Part 1

Image Versioning

- Mỗi image tagged với **Git SHA**:
 - `docker.io/{user}/zalord-auth:{sha}`
 - `docker.io/{user}/zalord-chat:{sha}`
 - ...
- Sau khi deploy thành công: promote `:latest` tag
- **Selective build**: chỉ rebuild service có file thay đổi → tiết kiệm thời gian CI
- Không có image nào bị overwrite — SHA tag immutable

Testing trong CI — Chưa có

[PLACEHOLDER]

CI/CD pipeline hiện tại: build + deploy.
Chưa có bước automated testing.

Nên bổ sung:

- Unit tests (JUnit 5 cho Java, Go test cho Go)
- Integration tests (Testcontainers)
- Contract tests giữa services

Versioning & Rollback Strategy (Req 15) — Part 2

Rollback

- K8s giữ revision history của Deployment
- `kubectl rollout undo deployment/auth-service -n zalord-dev` → về image SHA của commit trước
- `kubectl rollout status` trong pipeline → fail fast (180s) nếu pods không healthy

[PLACEHOLDER — chụp hình]

*Docker Hub repository
hiển thị tags theo SHA:
zalord-auth:abc1234
zalord-chat:abc1234
...*

Canary Deployment

Chiến lược Canary

- Khi có tính năng rủi ro cao, ta có thể triển khai **Canary release**.
- K8s cho phép chạy song song 2 bản Deployment (stable và canary) phía sau cùng 1 Service.
- Có thể dùng Kong API Gateway để chia traffic (ví dụ 90% stable, 10% canary) thay vì chia đều.

[PLACEHOLDER — sơ đồ canary]

Minh họa Service chia traffic vào bản v1 (90%) và v2-canary (10%)

Scenario 1: Circuit Breaker Activation (Req 16) — Scenario

Scenario: media-service xuống → CB open

- ❶ Kill media-service pod:
`kubectl delete pod media-service-xxx`
- ❷ Gửi message có attachment qua API
- ❸ Observe: **circuit breaker OPEN** sau >50% failures trong 10 calls (Resilience4j)
- ❹ Response: fallback method trả về error (không hang)
- ❺ Restart media-service pod (K8s tự restart)
- ❻ Observe: CB → HALF-OPEN → 3

Kết quả kỳ vọng

- Hệ thống không crash khi media-service lỗi
- Fallback response trả về ngay (không chờ timeout)
- Tự recover khi media-service lên lại

Scenario 1: Circuit Breaker Activation (Req 16) — Logs

[PLACEHOLDER — chụp hình/log]

Logs của message-service:

- *CB state: CLOSED → OPEN*
 - *Fallback called*
- *CB state: OPEN → HALF-OPEN*
- *CB state: HALF-OPEN → CLOSED*

*Hoặc Grafana metric panel cho
Resilience4j state transitions*

Scenario 2: Pod Crash & Self-Healing (Req 16)

Scenario: chat-service pod crash

- 1 Client A và B đang chat real-time
- 2 `kubectl delete pod chat-service-xxx -n zalord-dev`
- 3 K8s phát hiện pod gone → tạo pod mới (Deployment replicas=1)
- 4 Frontend WebSocket: **exponential backoff reconnect** (cap 15s, logic trong `websocket.ts`)
- 5 Pod mới Ready → clients reconnect thành công

[PLACEHOLDER — chụp hình]

```
kubectl get pods -n zalord-dev
```

*Chụp: chat-service pod
RESTARTS count = 1, Running*

*Frontend console log:
— WS disconnected
— Reconnecting in 1s...
— Reconnecting in 2s...
— WS connected*

Scenario 3: Outbox resiliency (Req 16)

Scenario 3: Outbox resiliency

- ❶ Tắt RabbitMQ tạm thời
- ❷ Gửi messages → lưu vào outbox_events table (atomic với message)
- ❸ Bật lại RabbitMQ
- ❹ **OutboxScheduler** (poll 3s) phát hiện unpublished events → publish
- ❺ Không mất message nào

[PLACEHOLDER]

*Grafana: HTTP request rate
dip khi pod đang restart,
recover sau reconnect*

Tổng kết (1/2) — Đã triển khai

Kiến trúc & Backend

- ✓ DB-per-service (6 PostgreSQL + Redis + MinIO)
- ✓ Transactional Outbox (auth, message services)
- ✓ CQRS (message-service InboxProjector)
- ✓ Event-driven (RabbitMQ/Kafka switchable)
- ✓ gRPC sync + Circuit Breaker (Resilience4j)
- ✓ JWT HS256 auth (không cần OAuth2/Keycloak)

Hạ tầng & Vận hành

- ✓ Kong API Gateway (JWT, rate-limit, CORS, Lua)
- ✓ ConfigMap + SOPS-encrypted Secrets
- ✓ Multi-stage Dockerfile, Docker Compose
- ✓ Kubernetes (k3s, Kustomize)
- ✓ CI/CD GitHub Actions → Docker Hub → GKE / k3s
- ✓ Prometheus, Grafana, Loki (Logging)
- ✓ OpenTelemetry + Jaeger (Tracing)
- ✓ Swagger/OpenAPI aggregation

Chưa triển khai / Không phù hợp

- Automated testing trong CI (chưa implement)
- ELK Stack — Thay bằng Loki nhẹ hơn cho K8s
- Saga Pattern — không có distributed transaction phức tạp
- OAuth2/Keycloak — không phù hợp: closed system

Xem chi tiết

EXPLANATION.md — giải thích kỹ lý do các requirement không áp dụng hoặc có thay đổi so với yêu cầu ban đầu.

**Cảm ơn thầy và các bạn
đã lắng nghe!**